```
  chain :for_incomplete_tasks_only do
    @incomplete = true
  end
end
```

The chain method takes an argument and a block, with the argument being the name of the method you want to chain. Any arguments to that method become arguments to the block—in this case the method has no arguments. Typically, inside a chained method you set instance variables, which are then referenced inside the match method to affect the match. The for_incomplete_tasks_only method sets a flag used to determine how to query the project for the size being tested.

The new method can then be chained onto the matcher:

model/04/spec/models/project_spec.rb

```
it "can calculate total size" do
  expect(project).to be_of_size(10)
  expect(project).not_to be_of_size(5)
end

it "can calculate remaining size" do
  expect(project).to be_of_size(5).for_incomplete_tasks_only
end
```

What You've Done

In this chapter you learned about strategies for dealing with Rails models and looked at a way of driving the TDD process to add a particular feature. Once the process is added, the next step is refactoring, and you looked at how to mitigate complexity, duplication, and missing abstractions. You also looked at testing Rails features such as associations and validations, and at how to create your own RSpec matchers if the built-in ones aren't expressive enough.

The guidelines discussed in this chapter should give you some direction as you test-drive new business logic in your Rails application. Every test you write, however, depends on some data to run. Getting useful data into the test cleanly and quickly turns out to be kind of complicated. In the next chapter you'll look at several ways of managing test data.